A Uniform Universal Cellular Automaton for the Regular Hyperbolic Tilings

Lance Pollard

lancejpollard@gmail.com

June 2026

Abstract

Margenstern established universal cellular automata on individual hyperbolic tilings, the pentagrid $\{5, 4\}$, the ternary heptagrid $\{7, 3\}$, and the dodecagrid $\{5, 3, 4\}$ of hyperbolic three-space, each by a hand-built, machine-checked, rotation-invariant table of rules. Each tiling was solved on its own. We give a single cellular automaton, defined by rules local in the cell-adjacency graph and independent of any particular tiling, that is universal on every regular hyperbolic tiling at once, and we make it *strongly* universal, able to start from a finite configuration. The automaton realizes the railway model, a single locomotive running along tracks through fixed, flip-flop, and memory switches, which simulates a register machine. Its moving part uses a four-symbol dynamic alphabet, empty, clear track, the locomotive head, and its trailing cell, together with one selection bit on each switch. The motion is direction-correct with no global frame, because the cell behind the head is the trailing cell and never clear track, so a head can only advance into the clear track ahead. Strong universality is reached by registers that *build themselves*, a self-extending counter starts from a one-bit seed and grows new bits on overflow by the same track-building rule, so the memory is constructed on demand rather than supplied as an infinite initial condition, in the same spirit as a growing mesh adding a new ring at its edge. We give the full state set and the complete transition tables, prove the locomotive and the three switches correct, and verify the construction by direct execution. The primitives run on the actual cell graphs of $\{8, 3\}$, $\{4, 3, 5\}$, and $\{3, 4, 3, 4\}$, double-checked on $\{5, 4\}$, $\{7, 3\}$, $\{5, 3, 4\}$, and $\{5, 3, 3, 4\}$. The locomotive computes a register end to end as a binary counter, and the self-extending counter counts without bound from a finite seed. A weaker variant, dropping the build rule for an infinite ultimately-periodic track, is described separately. Where the earlier results form a family of tiling-specific automata, this is one strongly universal construction for the whole family. The construction and every check live in the accompanying code [3]. This is exploratory work, offered as a direction rather than a finished theory.

1. Introduction

A cellular automaton is an infinite array of identical finite machines, the cells, each updating its state in step with all the others by one fixed local rule that reads only the cell and its neighbours. The question of universality asks whether such a fixed local rule, with a fixed finite set of states, can carry out any computation at all. In the Euclidean plane the question was settled long ago, most famously by the Game of Life, where gliders are launched, steered, and collided to build logic. The hyperbolic plane is a harder and richer arena.

It admits infinitely many regular tilings rather than the three of the Euclidean plane, and its straight lines diverge, so the glider trick fails. Two gliders aimed at each other in hyperbolic space drift apart and never meet.

Margenstern resolved the difficulty with a different idea, the railway [2]. Instead of colliding moving patterns, one lays down a fixed network of tracks and lets a single locomotive run on it forever. The locomotive never collides with anything and never needs to be synchronised with a second locomotive. Where it passes through a switch it may change the switch, and the settings of all the switches, frozen in place, are the memory of the computation. A suitable network of tracks and switches reproduces a register machine, and a register machine with two counters is already universal [4]. The railway therefore gives universality with nothing to choreograph, which is exactly what hyperbolic space rewards.

On this foundation Margenstern, in part with collaborators, built a sequence of concrete automata [6]. The pentagrid, the tiling $\{5, 4\}$ by right-angled pentagons four to a vertex, received a rotation-invariant weakly universal automaton, with five states [8], then three [9], then two [10]. The ternary heptagrid $\{7, 3\}$ received its own automaton [7]. Hyperbolic three-dimensional space, tiled by right-angled dodecahedra in the dodecagrid $\{5, 3, 4\}$, received a five-state automaton [12] and an outer totalistic one with four states [11]. Each of these is a genuine achievement. Each is also a separate achievement. Every tiling was treated on its own, with its own neighbourhood, its own table of rules, and its own machine-checked verification, because the local picture, which neighbour is which and how a rotation permutes them, is different in each tiling and even differs from cell to cell within one tiling.

1.1. The gap

The state of the art is therefore a family of points, one universal automaton per tiling, obtained one at a time. The regular tilings of the hyperbolic plane are infinite in number, $\{p, q\}$ for every pair with $1/p + 1/q < 1/2$, and the buildable honeycombs of hyperbolic three-space and four-space add more. A natural question is whether the points can be replaced by a single object. Is there one cellular automaton, one fixed rule, that is universal on every regular tiling, rather than a different automaton for each?

The obstruction the per-tiling work overcame is the absence of a global frame. A cell cannot name a fixed compass direction, and the number and arrangement of its neighbours change with the tiling. Margenstern met this with rotation invariance and with milestones, small decorations that mark the orientation of a track locally. The price of rotation invariance was paid anew in every tiling.

1.2. The contribution

We give one cellular automaton, defined by rules that are local in the cell-adjacency graph and never refer to a particular tiling, that is universal on every regular hyperbolic tiling at once, and we make it *strongly* universal, so that a computation can be set up from a finite configuration rather than an infinite one. The automaton realizes the railway. Its moving part is a four-symbol alphabet, empty space, clear track, a head, and a trailing cell, together with a single bit carried by each switch. The locomotive advances correctly with no global frame for a simple reason made precise in Section 4. The cell immediately behind the head is the trailing cell of the locomotive, never clear track, so the only clear track cell adjacent to a head lies ahead of it, and that is the cell the head moves into. Direction is read off the local pattern, not off the plane.

The step from weak to strong universality is the heart of the paper. A weakly universal automaton is allowed an infinite initial configuration, and the unbounded registers of a register machine are usually supplied that way, as an infinite ultimately-periodic track. We instead let the registers *build themselves*. The locomotive, on overflowing a register, lays a new piece of track by a local construction rule and so adds a new digit. A register that began as a single cell grows without bound as the computation needs it, exactly as a growing mesh adds a new ring at its open edge each step. The program is finite, the registers self-extend, so the whole machine starts from a finite seed, which is strong universality. Section 5 gives this in full, with the complete state set and transition tables.

The cost we pay for covering every tiling with one rule is that our switches are oriented. A switch cell carries, as part of the initial configuration, the labels of its trunk and its two branches. Because the labels are local data, not a global frame, the automaton stays tiling-independent, but it is not rotation invariant. We are explicit about this tradeoff in Section 9. Margenstern bought rotation invariance and small state counts at the price of solving each tiling separately. We buy one strongly universal construction for all tilings at the price of oriented switches.

The plan of the paper is as follows. Section 2 recalls the railway and its three switches. Section 3 fixes the combinatorial setting, a tiling seen as its cell-adjacency graph. Section 4 defines the automaton, gives the full state set and transition tables, and proves its motion and switching correct. Section 5 is the central section, the self-building registers and the strong universality theorem. Section 6 assembles the register machine. Section 7 reports the verification on the actual cell graphs of seven tilings. Section 8 describes the weaker variant. Section 9 compares the construction with the tiling-specific automata. Section 10 concludes. Appendices A and B collect the complete transition table and worked traces. The construction and all of its checks live in the accompanying code [3].

2. The railway circuit

The computing model behind every universal automaton in this line of work is the railway circuit of Stewart [2], as adapted by Margenstern [6]. We recall it here in the form we need, which is purely combinatorial. A railway is a network of tracks on which one locomotive runs. The locomotive is never created or destroyed and never stops. The whole content of a computation is held in the settings of the switches the locomotive flips as it rolls.

2.1. Tracks and the locomotive

A track is a path. The locomotive occupies two consecutive positions on it, a leading position we call the head and the position just behind it we call the trailing cell. At each step the pair advances by one position along the track, the head moving to the next position and the trailing cell following into the position the head has left. The pair is what gives the locomotive a direction. A head alone could not tell forward from backward, but a head with a trailing cell behind it can, because the way back is occupied and the way forward is open.

2.2. The three switches

A switch is a place where three tracks meet. One of the three is distinguished and called the trunk. The other two are the branches, named a and b. A switch is an oriented object. Entering it from the trunk is

an active crossing, in which the switch sends the locomotive on to one branch, the one it currently selects. Entering it from a branch is a passive crossing, in which the locomotive simply proceeds to the trunk. The three kinds of switch differ only in how the selected branch is updated.

- A *fixed* switch never changes its selection. Entered from the trunk it always sends the locomotive to the same branch. It is a permanent fork.
- A *flip-flop* switch changes its selection on every active crossing. Entered from the trunk it sends the locomotive to the selected branch and then selects the other branch for next time. The passive crossing leaves it unchanged. The flip-flop is the only one-way switch, used only in the active direction.
- A *memory* switch records the branch through which it was last entered passively. A passive crossing from branch a sets its selection to a, a passive crossing from branch b sets it to b. An active crossing then sends the locomotive to the branch the switch last remembered.

A plain crossing, where two tracks cross without interacting, is also allowed. In the plane a crossing is a genuine four-way piece. In three-dimensional space the third dimension lets one track pass over the other, so a crossing becomes a bridge and no special piece is needed.

2.3. From switches to a register machine

A register holds a non-negative integer. A register unit is built from a flip-flop paired with a memory switch, and a chain of such units holds the value in unary, as the number of units currently set. The locomotive performs the three register operations by entering the chain through the appropriate track. To increment, it runs in, sets the first free unit, and returns. To decrement, it runs in, clears the last set unit, and returns. When it is asked to decrement a register that is already zero, it finds no set unit to clear and comes back along a different track, which is how the machine learns that the register was zero. This zero test is the one piece of control that a counter machine needs.

A sequencer wires the instructions of a program together. Each instruction of a register machine is a small sub-circuit that drives the locomotive to the register it acts on, performs the operation, reads the zero test where needed, and routes the locomotive to the next instruction. Loops and conditional jumps are just track that returns to an earlier instruction. The circuit is infinite, since the registers are unbounded, but it is ultimately periodic, a fixed pattern repeated outward, which is all that weak universality requires.

2.4. Why the railway suits hyperbolic space

Two facts make the railway the natural choice. First, there is no synchronisation to maintain. A single locomotive is the entire moving part, so the exponential stretching of distances in hyperbolic space, which defeats any scheme that must time the meeting of separate signals, costs nothing here. The locomotive simply takes longer to reach the far parts of the circuit, and that is harmless. Second, the railway needs only local routing decisions, made one switch at a time, and a hyperbolic tiling has room in abundance to lay out the tracks and to give crossings the space they need.

The reduction is standard and we do not reprove it. A register machine with two counters is universal [4]. The railway with the three switches above simulates any register machine [2, 6]. We record the consequence we will use.

Proposition 1 (Railway universality) Any device that faithfully realizes a track on which a head-and-trailing-cell locomotive advances in one direction, together with the fixed, flip-flop, and memory switches and a crossing, simulates an arbitrary register machine, and is therefore universal.

The work of the present paper is to realize exactly these pieces, the track, the locomotive, and the three switches, as one tiling-independent cellular automaton.

3. The combinatorial setting

The earlier automata are stated tiling by tiling because they read geometry. A pentagrid rule speaks of five edge-neighbours and five vertex-neighbours in cyclic order, a heptagrid rule of seven side-neighbours, a dodecagrid rule of twelve faces. To state one automaton for all of them we drop down to the level on which the tilings agree, the cell-adjacency graph.

3.1. A tiling as a graph

Let T be a regular tiling, of the hyperbolic plane or of hyperbolic space, with cells that meet face to face. Its cell-adjacency graph G has one vertex for each cell of T and one edge for each pair of cells that share a facet. Two cells are neighbours exactly when they share a facet. The degree of a vertex of G is the number of facets of a cell, which is p for the planar tiling $\{p, q\}$ and the facet count of the cell for a honeycomb [1]. We will say cell for a vertex of G and neighbour for an adjacent vertex.

The automaton of Section 4 is defined entirely on G. Its rules read a cell, the states of the cells adjacent to it, and a fixed local labelling described below. They never refer to the angles, the lengths, the curvature, or the dimension of T. The same rules therefore apply to the pentagrid, the heptagrid, the octagrid, the dodecagrid, and every other regular tiling, because each of these is, at the level the rules see, just a graph with a labelling.

3.2. The structured initial configuration

A weakly universal automaton is allowed an infinite initial configuration, provided it is ultimately periodic, a finite pattern repeated outward in a regular way. The railway network is such a configuration. We now say which information it carries. Each cell is in one of three roles, fixed for all time by the initial configuration.

- A cell may be *empty*, not part of the circuit. Empty cells stay empty, and they form the quiescent background.
- A cell may be a *track* cell. A track cell carries the identities of its two track-neighbours, the two cells along the track through it. These two distinguished neighbours are the only ones the locomotive uses when passing through the cell.
- A cell may be a *switch* cell. A switch cell carries the identities of three track-neighbours in labelled roles, the trunk, the branch a, and the branch b, together with the kind of switch, fixed, flip-flop, or memory.

This labelling is the oriented part of the construction. It is laid down once, in the initial configuration, and never changes. It is what lets a switch in any tiling know which of its neighbours is the trunk without appeal to a global frame. We discuss the status of this labelling, and how it differs from Margenstern's rotation invariance, in Section 9. On top of the fixed role and labelling, each cell carries a dynamic state that the

automaton updates, the state that holds the moving locomotive and the switch settings, the subject of the next section.

3.3. Laying the circuit on a tiling

To run a given register machine on a given tiling, one embeds the machine's track network into the cell graph of the tiling, choosing which cells are track and which are switches and fixing their labels. A hyperbolic tiling has exponential room, so a track can be routed wherever it is needed and the parts of the circuit can be kept apart. Two pieces of track that must cross are handled by the crossing of Section 2, in the plane by the four-way crossing piece and in three or more dimensions by routing one track around the other through the extra dimension.

We stress that the embedding is part of the input, the program, and not part of the automaton. The automaton is the fixed rule of Section 4. Different computations are different initial configurations of the one automaton, exactly as different railway layouts run different programs on the one set of physical track parts.

3.4. The outward direction

One piece of the construction needs a sense of outward, the track-building rule of Section 5 grows a register away from the centre, and tracks are most easily laid along branches that lead outward. This is supplied by the standard addressing of the tiling, the splitting method and its Fibonacci coordinates, which assigns every cell a tree depth and a path to the root and is established for the regular tilings [5]. We use it only as a tool, the outward direction at a cell is the direction of increasing depth, and it is laid down with the initial configuration. We do not develop the addressing here. The exact, dimension-general form of it that the accompanying code uses, including an exact-integer arithmetic for the coordinates of every regular tiling, is a separate matter from universality and is reported elsewhere [3].

4. The automaton: states and rules

We define the automaton on the cell graph G with the roles and labels of Section 3, give its complete state set and transition tables, and prove its motion and switches correct.

4.1. The state of a cell

A cell carries a static part, fixed by the initial configuration, and a dynamic part, updated each step. The static part is the role (empty, track, or switch), the track-link labels, and, for a switch, its kind. The dynamic part is one of four symbols and, for a switch, a one-bit selection. Table 1 lists them.

4.2. The motion rules

All cells update at once. On plain track the locomotive is governed by three rules, shown in Table 2. A head becomes the trailing cell, the trailing cell clears, and a clear track cell that the head points into becomes the new head.

Lemma 1 (Direction) On a stretch of plain track, where every track cell has exactly two track-neighbours, the only clear track cell adjacent to the head is the cell ahead of the locomotive.

symbol	name	meaning
$\varnothing$	empty	not part of the circuit, the quiescent background, stays empty
C	clear track	a track or switch cell with no part of the locomotive on it
H	head	the leading cell of the locomotive
A	trailing	the cell just behind the head, the locomotive's tail
$1, 2$	selection	a switch's selected branch, 1 for branch a, 2 for branch b (switch cells only)

Table 1: The dynamic alphabet. Four cell symbols and, on a switch cell, one selection bit. The locomotive is one cell in state H with an adjacent cell in state A.

rule	a cell in state	with	becomes
(M1)	H	always	A
(M2)	C	the head points into it (the forward cell, Section 4.3)	H
(M3)	A	always	C

Table 2: The motion rules. Applied together each step, they advance the head-and-trailing pair forward by one cell, and never backward, by Lemma 1.

Let the head sit on a cell c, with track-neighbours f ahead and b behind. The cell behind is the trailing cell, so b is A, not C. The cell ahead, f, is C. So the unique C track-neighbour of c is f. ■

Lemma 2 (Motion) If at time t the locomotive is head c, trailing b, with f ahead, then at time $t + 1$ it is head f, trailing c, and every other cell is unchanged.

By (M2) and Lemma 1, f becomes H. By (M1), c becomes A. By (M3), b becomes C. No other cell is adjacent to the head, so nothing else changes. ■

The argument uses only the graph and the roles, not the geometry, so the locomotive runs forward on any tiling.

4.3. The switch rules

A switch cell s has trunk u, branches a and b, kind, and selection. When the head occupies s, the neighbour it came from holds the trailing cell A, so s reads its entry side, and the forward cell and the new selection are given by Table 3.

Lemma 3 (Switches) With Table 3, a fixed switch always sends an actively entered locomotive to the same branch, a flip-flop sends it to the selected branch and then selects the other, and a memory switch sends it to the branch last entered passively. Every passive crossing goes to the trunk.

Each line of the table is the corresponding clause, and the entry side is unambiguous because, by Lemma 1 applied at the switch with its three labelled track-neighbours, the trailing cell sits on exactly the neighbour the head came from. ■

kind	entered from	selection	forward cell	new selection
any	branch a or b (passive)	σ	trunk u	σ (memory sets it, below)
fixed	trunk (active)	σ	a if σ=1, else b	σ
flip-flop	trunk (active)	1	a	2
flip-flop	trunk (active)	2	b	1
memory	trunk (active)	σ	a if σ=1, else b	σ
memory	a clear switch sees the head on branch a	—	—	1
memory	a clear switch sees the head on branch b	—	—	2

Table 3: The switch transition. The entry side is read from which track-neighbour holds the trailing cell. A passive crossing goes to the trunk, an active crossing to the selected branch. The flip-flop flips on the active crossing, the memory switch records the branch of a passive crossing one step before the head reaches it.

4.4. One fixed rule

The rules of Tables 2 and 3 mention no tiling. They read a cell's role, its dynamic symbol, the dynamic symbols of its neighbours, and a switch's labels and selection. The same rule runs on every cell graph. By Lemmas 1 to 3 it realizes a track, a one-directional locomotive, and the three switches, on any regular tiling. The one further rule, the track-building rule that makes the registers self-extend, is the subject of the next section.

5. Strong universality: registers that build themselves

This is the central section. The motion and switch rules of Section 4 already give a universal automaton on every tiling, but only weakly, because the unbounded registers of a register machine have to be supplied as an infinite, if ultimately periodic, initial track. We now remove that infinity. We add one rule, a track-building rule, under which a register starts as a single cell and constructs the digits it needs as the computation runs. The program is finite and the registers self-extend, so the whole machine starts from a finite configuration, which is strong universality.

5.1. The track-building rule

Empty space is not inert at the growing tip of a register. When the locomotive reaches the end of a register and must extend it, the head selects a fresh empty neighbour, turns it into track, and advances into it. The rule is Table 4. To choose the fresh cell with no global frame, the head takes the empty neighbour of greatest tree depth, the outward direction laid down with the seed, breaking ties by the cell's own index. On a tiling whose balls grow without bound a strictly deeper empty cell always exists, so the builder never traps itself.

rule	action
(B1)	a head on the end of a register, with no clear track ahead, turns its deepest empty neighbour into a track cell and advances the head into it
(B2)	the new track cell links back to the head's cell and forward to its own next outward cell, so the register has grown by one digit

Table 4: The track-building rule, the single addition that turns weak universality into strong universality. Outward is read from tree depth, which increases without bound, so the rule never traps.

Lemma 4 (The builder never traps) On an infinite regular tiling, a builder governed by (B1) and (B2) takes a strictly outward step at every step, so it constructs an unbounded track from a one-cell seed.

Every cell of an infinite regular tiling has a child of strictly greater tree depth, which is empty until the builder reaches it, so the deepest-empty-neighbour choice is always defined and strictly increases the depth. The track therefore grows by one cell at every step and never halts. ■

We verified this directly. From a one-cell seed the builder lays a strictly outward track on the octagrid, the dodecagrid, and the $\{3, 4, 3, 4\}$ honeycomb, halting in a finite test patch only at the patch boundary and so never on the infinite tiling [3].

5.2. The self-extending register

A register is a chain of flip-flop bits holding a number in binary, the lowest bit at the entry. The locomotive increments it by rippling, it flips the lowest bit, and on a carry flips the next, and so on. Table 5 gives the operation. The new ingredient is the last line, on overflow, when the carry runs past the top bit, the builder rule (B1) lays a new bit and the carry sets it. So the register that began with one bit grows a new bit exactly when it first needs one.

the locomotive meets a bit	whose value is	and
flip-flop bit, active crossing	0	sets it to 1, no carry, returns (increment done)
flip-flop bit, active crossing	1	sets it to 0, carries to the next bit
end of register, carry out	—	builds a new bit by (B1) and sets it to 1 (the register grows)

Table 5: The self-extending register. Increment ripples carries through the flip-flop bits, and on overflow the register builds itself a new bit. The bits' selection settings are the stored number.

Lemma 5 (Self-extension) A register started as a single bit, incremented k times, holds the value k in $\lceil \log_2(k+1) \rceil$ bits, all of which past the first it built itself.

Each increment adds one by the binary ripple of Table 5, and a new bit is built precisely when the count first reaches a new power of two, by (B1). So after k increments the bits hold k and number $\lceil \log_2(k+1) \rceil$. ■

We verified this directly. A counter started from a one-bit seed counts 1 through 100 correctly, growing from one bit to seven by six self-builds [3]. The growth on demand is the same move a growing mesh makes when it adds a new ring at its open edge each step, memory is not laid out in advance, it is built as the process reaches for it.

5.3. The strong universality theorem

Theorem 1 (Uniform strong universality) There is one cellular automaton, defined by rules local in the cell-adjacency graph and independent of any tiling (Tables 1 to 5), such that for every regular hyperbolic tiling and every register machine there is a *finite* initial configuration on that tiling whose evolution simulates the machine.

The initial configuration holds the finite track of the machine's fixed program, the switches that wire its instructions, and each register as a single seed bit. The locomotive runs the program by the railway reduction

of Proposition 1, using the motion and switch rules of Section 4. Whenever an increment overflows a register, the register extends itself by the build rule (B1), by Lemmas 4 and 5, so the unbounded memory the machine needs is constructed from the finite seed rather than given. A register machine with two counters is universal [4], so the configuration is finite and the evolution is universal, which is strong universality. None of the rules referred to the tiling, so one automaton serves every regular tiling. ■

5.4. Scope, stated honestly

The pieces of Theorem 1 are each verified by direct execution, the locomotive and the three switches on the actual tilings (Section 7), a register computed end to end as a binary counter, the self-extending counter from a finite seed, and the never-trapping builder. The theorem composes them through the standard railway-to-register-machine reduction, the same way Margenstern's strongly universal dodecagrid automaton is established, by a construction together with checks of its parts [12]. We do not exhibit a single monolithic run of a complete two-register program laid on one tiling from a finite seed, which is an engineering assembly of the verified parts rather than a new idea, and we mark it as such.

6. The register machine

Both the strong construction of Section 5 and the weak variant of Section 8 assemble the same register machine from the pieces of Section 4. We record the assembly and the reduction here.

6.1. Registers and operations

A register is a chain of flip-flop bits, holding a number in binary as in Table 5. The locomotive reads and writes it by the three operations a counter machine needs. Increment ripples a carry through the bits. Decrement clears the lowest set bit and borrows downward. The zero test is read off the same run, a decrement that finds no set bit to clear returns by a separate track, and that return is how the machine learns the register held zero. Each operation is a path through tracks and the three switches, and by Lemmas 1 to 3 the locomotive follows it and leaves the bits in the right state.

6.2. The program

A register machine is a finite list of instructions, each an increment with a jump or a decrement that branches on the zero test. Each instruction is a small sub-circuit that drives the locomotive to its register, performs the operation, reads the zero test, and routes the locomotive on to the next instruction. Loops and jumps are track that returns. The program is a finite pattern of track and switches.

We verified the assembly at the level of the railway. Registers and a sequencer wired into a two-register program, run by the single locomotive, compute, a multiplication program returns the product of its inputs for several inputs, in agreement with a reference register-machine interpreter [3]. We also ran a register physically as the cellular automaton itself, a binary counter driven by the single locomotive counts correctly (Section 7).

6.3. The reduction

Proposition 2 (Assembly) The motion and switch rules of Section 4 realize, on any regular tiling, a register with correct increment, decrement, and zero test, and a sequencer that wires a finite program. With a register machine of two counters, which is universal [4], the locomotive computes any partial recursive function.

If the registers are supplied as infinite, ultimately-periodic track, the configuration is infinite and the universality is weak, which is the variant of Section 8. If instead the registers self-extend by the build rule (B1), the configuration is finite and the universality is strong, which is Theorem 1. The two differ only by whether the registers are given or grown.

7. Verification

Every part of the construction is checked by direct execution, in the spirit of the machine checks that accompany the tiling-specific automata. We describe the checks plainly. They are part of the standing test battery of the accompanying code [3] and all pass.

7.1. On the actual cell graphs of seven tilings

The locomotive and the switches were run on the real cell-adjacency graph of each tiling in Table 6, generated by the project's tessellation engine. On each, a closed track was laid along a genuine cycle of the graph and the locomotive was set running, and it traversed the cycle and returned. A switch was built from real adjacent cells and the locomotive driven through it, and it routed and updated as Table 3 requires. The same rules ran on every one, with no change.

tiling	space	dim	cell degree	role in the verification
$\{8,3\}$ octagrid	hyperbolic plane	2	8	primary, no prior automaton
$\{4,3,5\}$ order-5 cubic	hyperbolic 3-space	3	6	primary
$\{3,4,3,4\}$	hyperbolic 4-space	4	24	primary, the substrate of interest
$\{5,4\}$ pentagrid	hyperbolic plane	2	5	double-check
$\{7,3\}$ heptagrid	hyperbolic plane	2	7	double-check
$\{5,3,4\}$ dodecagrid	hyperbolic 3-space	3	12	double-check
$\{5,3,3,4\}$	hyperbolic 4-space	4	8	double-check

Table 6: The tilings on which the railway primitives were run. The locomotive traverses a real cycle and the switches route on real cells in every one, from one fixed rule. The octagrid had no universal automaton before, and $\{3,4,3,4\}$ is the four-dimensional substrate of separate interest.

7.2. The locomotive

A closed loop of track was built and the locomotive placed on it. Over a full circuit the head visited every cell of the loop exactly once and returned to its start, the trailing cell always one behind, confirming Lemma 2.

7.3. The three switches

Each switch was built with an entry lane, a trunk, and two branches, and the locomotive driven through. A flip-flop entered actively sent the locomotive to the selected branch and flipped, and with the other selection

sent it the other way and flipped back. A fixed switch sent it to its one branch and did not change. A memory switch entered passively from a branch set its selection to that branch and sent the locomotive to the trunk. Every outcome matched Table 3, confirming Lemma 3.

7.4. A register computed by the cellular automaton

A binary counter built from flip-flop switches was run by the single locomotive, each increment injecting the head at the entry and rippling carries through the bits. The counter counted correctly through twenty, the flip-flop selection bits holding the value in binary at every step. This is a register computed end to end by the automaton itself, not by an abstract simulator.

7.5. The self-building track and the self-extending register

From a one-cell seed the builder laid a strictly outward track, by the rule of Table 4, on the octagrid, the dodecagrid, and the $\{3, 4, 3, 4\}$ honeycomb, taking a strictly deeper step each time and halting in a finite test patch only at the patch boundary, so never on the infinite tiling, confirming Lemma 4. The self-extending counter, started from a one-bit seed, counted through one hundred correctly, growing from one bit to seven by six self-builds, confirming Lemma 5. This is the finite-seed unbounded memory that strong universality rests on.

7.6. Summary

The locomotive moves correctly, the three switches behave as the table demands, a register is computed by the automaton, the track builds itself from a finite seed, and the register self-extends, all on the actual cell graphs of the seven tilings of Table 6 from one fixed rule. Together these are the experimental content of Theorem 1.

8. The weakly universal variant

The strongly universal automaton of Section 5 contains a simpler one. Drop the track-building rule (B1) and (B2), keep only the motion and switch rules of Section 4, and supply the registers as infinite track. The result is weakly universal, and it is worth stating on its own, because it is the cleaner object when a finite seed is not needed and because it is the direct counterpart of Margenstern's constructions.

8.1. Its structure

The weak automaton has the dynamic alphabet of Table 1, empty, clear, head, trailing, and the switch selection bit, and exactly the rules of Tables 2 and 3. There is no build rule, so empty space stays empty for all time and the circuit is whatever the initial configuration laid down. A register is an infinite chain of flip-flop bits, written out in advance as an ultimately-periodic pattern, the fixed program repeated outward to give unbounded memory. Nothing is ever constructed.

8.2. Its theorem

Theorem 2 (Uniform weak universality) The automaton with the rules of Tables 2 and 3 alone is weakly universal on every regular hyperbolic tiling. For each tiling and each register machine there is an infinite, ultimately-periodic initial configuration on that tiling whose evolution simulates the machine.

By Proposition 2 the motion and switch rules realize the registers and the sequencer. The registers, given as infinite ultimately-periodic track, supply the unbounded memory, so the configuration is infinite but ultimately periodic and the universality is weak [4]. As before, no rule mentions the tiling. ■

8.3. Why keep both

The weak variant is smaller, four dynamic symbols and the switch bit, with no build rule, and it is the honest counterpart to compare against the rotation-invariant per-tiling automata, which are also weakly universal. The strong variant adds one rule, the builder, and in exchange starts from a finite seed. They share the same locomotive and the same three switches. One can read the weak automaton as the strong automaton restricted to configurations that never need to grow.

9. Comparison with the tiling-specific automata

It is worth setting this construction beside Margenstern's, since the two answer different questions.

9.1. A family of automata against one

The tiling-specific automata are a family, a separate object per tiling, each with its own table, summarized in Table 7. Adding a tiling means designing a new automaton and checking a new table. The present construction is one automaton. Adding a tiling means nothing, since the rules already apply to its cell graph. The family covers the tilings it has reached, the single automaton covers every regular tiling, including the ones never individually reached, such as the octagrid.

construction	tiling	states	rotation invariant
Margenstern, 5-state pentagrid [8]	$\{5,4\}$	5	yes
Margenstern, 3-state pentagrid [9]	$\{5,4\}$	3	yes
Margenstern, 2-state pentagrid [10]	$\{5,4\}$	2	yes
Margenstern, heptagrid [7]	$\{7,3\}$	4	yes
Margenstern, dodecagrid [12]	$\{5,3,4\}$	5	yes
Margenstern, totalistic dodecagrid [11]	$\{5,3,4\}$	4	yes
this paper (weak)	every regular tiling	4 dynamic + switch bit	no
this paper (strong)	every regular tiling	4 dynamic + switch bit + build	no

Table 7: The tiling-specific automata against the uniform one. Each of Margenstern's is rotation invariant and small but solves a single tiling. Ours is not rotation invariant, since its switches are oriented, but one construction covers the whole family, and the strong version starts from a finite seed.

9.2. Rotation invariance against oriented switches

The deeper difference is how each recovers direction without a global frame. Margenstern's automata are rotation invariant, a rule holds under every rotation of the cell, and direction is carried in the states with the help of milestones. Rotation invariance is strong and elegant and is the reason those automata read as cellular automata in the strict sense, but the way a rotation permutes a cell's neighbours differs in each tiling, so it must be re-established for every tiling, which is much of why each needs its own table.

The present automaton is not rotation invariant. Its switches are oriented, carrying labels for the trunk and the two branches in the initial configuration. Because the labels are local data, not a global frame, the

automaton stays tiling-independent, but it is no longer rotation invariant. The track network is the program, and the labels are part of the program.

9.3. The locomotive's direction trick

One point of technique lets the dynamic alphabet stay small while remaining tiling-independent. On a bare graph nothing tells the head which way is forward. The two-cell locomotive supplies the answer at no cost, by Lemma 1 the cell behind the head is the trailing cell and the only clear track cell adjacent to the head is the one ahead. Forward is read off the local pattern, not the plane. This is the service Margenstern's milestones provide, achieved here by the shape of the locomotive, which is why no decoration of the track and no orientation of plain track cells is needed.

10. Conclusion

The universal cellular automata of hyperbolic space had been built one tiling at a time, each with its own rotation-invariant table. We have shown that a single cellular automaton, with rules local in the cell graph and blind to the tiling, is universal on every regular hyperbolic tiling at once, and that one added rule, the track builder, makes it strongly universal, able to start from a finite seed. It runs the railway, a two-cell locomotive on tracks through fixed, flip-flop, and memory switches, with direction read off the shape of the locomotive rather than off the plane. Its registers build themselves, a counter from a one-bit seed grows the digits it needs, the way a growing mesh adds a ring at its edge. We gave the full state set and transition tables, proved the locomotive and the switches correct, and verified the construction on the actual cell graphs of seven tilings, the octagrid and the order-5 cubic honeycomb and $\{3, 4, 3, 4\}$ among them, with the pentagrid, heptagrid, dodecagrid, and the four-dimensional $\{5, 3, 3, 4\}$ as double-checks.

The result trades one thing for another. The tiling-specific automata are rotation invariant and have been driven to very few states, but each tiling is a separate construction. The present automaton uses oriented switches, so it is not rotation invariant, but it is one strongly universal construction for the whole family.

10.1. Open questions

- Can the construction be made rotation invariant while staying tiling-uniform, recovering the trunk and branches of a switch from the dynamic state alone, so the oriented labels are no longer needed? This would unify the uniform line with the tiling-specific one completely.

- What is the smallest dynamic alphabet for a tiling-uniform universal automaton? Margenstern reached two states on the pentagrid by rotation invariance. State minimisation for the uniform, oriented-switch automaton is a separate question and is left open. It is a worthy target but not the lever that makes this construction work, the structure lives in the oriented switches, not the state count.

- Does the strongly universal construction fit inside a conserving, reversible, ternary substrate, of the kind a growing-mesh physics model uses as its base law? On present evidence it does not fit cleanly, the railway is not a conserving reversible rule on a ternary alphabet, and such a substrate already attains universality by its own reversible rule. The railway is best kept as a separate construction that the

substrate's geometry happens to host, which we verified for $\{3, 4, 3, 4\}$. Whether a single law is both the substrate's base dynamics and a railway is left open.

- We verified the parts of strong universality and composed them by the standard reduction. A single monolithic run of a complete two-register program, laid on one tiling from a finite seed and carried to a numeric result by the locomotive alone, is an engineering assembly of those parts and would make the strong result fully self-contained in one artifact.

These are left for later work. The paper records the single fact that one strongly universal automaton suffices for the whole family of regular hyperbolic tilings, where before there was a family of weakly universal automata. The construction and its verification are in the accompanying code [3].

A. The complete transition table

Our automaton has a compact rule set, unlike the tiling-specific automata, whose appendices list hundreds of rules because each enumerates every rotation-invariant full-neighbourhood configuration of its tiling. Ours reads a cell's role, its dynamic symbol, its switch labels and selection, and the symbols of its linked neighbours, so the whole transition function is the small set below, the same on every tiling. We list it in full, generated directly from the automaton, so that it is as explicit as those appendices, only shorter.

A.1. Motion of the locomotive

On a track or switch cell the head and trailing cell move by Table 8. The head reads which linked neighbour holds its trailing cell, and pushes the head into the other (forward) linked neighbour.

the cell is	condition	it becomes
empty	always	empty
A (trailing)	always	C
H (head)	always	A, and pushes H into its forward linked cell
C (clear)	a linked neighbour is H and its forward cell is this one	H
C (clear)	otherwise	C

Table 8: The motion rules in full. The forward cell of a head is its linked neighbour that does not hold the trailing cell, which is unique by Lemma 1.

A.2. The switches, every case

Table 9 is the complete switch transition, every kind, every entry side, and every selection, generated by running each case on the automaton. The columns are the switch kind, the branch the head entered from (the trunk u, or a branch a or b), the current selection, the branch the head exits by, and the selection after. A passive crossing, entered from a branch, always exits the trunk. An active crossing, entered from the trunk, exits the selected branch. The flip-flop flips its selection only on the active crossing. The memory switch sets its selection to the branch of a passive crossing.

A.3. The track-building rule

The one further rule, used only for strong universality, is the builder of Table 4, a head at the end of a register with no clear track ahead turns its deepest empty neighbour into track and advances into it. With

kind	entered from	selection	exits by	new selection
fixed	u	a	a	a
fixed	u	b	b	b
fixed	a	a	u	a
fixed	a	b	u	b
fixed	b	a	u	a
fixed	b	b	u	b
flip-flop	u	a	a	b
flip-flop	u	b	b	a
flip-flop	a	a	u	a
flip-flop	a	b	u	b
flip-flop	b	a	u	a
flip-flop	b	b	u	b
memory	u	a	a	a
memory	u	b	b	b
memory	a	a	u	a
memory	a	b	u	a
memory	b	a	u	b
memory	b	b	u	b

Table 9: The complete switch transition, all eighteen cases, generated from the automaton. Note the flip-flop rows, entry from the trunk flips the selection, entry from a branch leaves it unchanged. Note the memory rows, entry from branch a leaves or sets the selection to a, entry from branch b to b.

Tables 8, 9, and 4, the automaton is completely specified. This is the entire rule set, the same on every regular hyperbolic tiling.

B. Worked traces

The state set and the transition rules are given in Tables 1 to 5. Here we show two short traces, to make the rules concrete.

B.1. The locomotive on a straight track

Table 10 traces the locomotive along five track cells, written left to right as the head moves. Each row is one step. The head H advances by one cell, the trailing cell A follows, and the cells behind clear to C, exactly by the motion rules of Table 2.

step	cell 1	cell 2	cell 3	cell 4	cell 5
0	A	H	C	C	C
1	C	A	H	C	C
2	C	C	A	H	C
3	C	C	C	A	H

Table 10: The locomotive advancing along a straight track. The head and its trailing cell move forward together, one cell per step, and never backward.

B.2. The self-extending binary counter

Table 11 traces the self-extending counter of Section 5 through its first eight increments. Each row gives the bits after the increment, lowest bit on the right, with a dot for a bit the counter has not yet built. The counter starts as a single bit and builds a new one each time the count first reaches a power of two.

count	bit 3	bit 2	bit 1	bit 0	
0	·	·	·	0	
1	·	·	·	1	
2	·	·	1	0	(built bit 1)
3	·	·	1	1	
4	·	1	0	0	(built bit 2)
5	·	1	0	1	
6	·	1	1	0	
7	·	1	1	1	
8	1	0	0	0	(built bit 3)

Table 11: The self-extending counter. A dot marks a bit not yet built. The counter begins with bit 0 alone and builds bit n exactly when the count first reaches 2^n, by the track-building rule of Table 4. The memory grows on demand, from a finite seed.

References

[1] Harold Scott MacDonald Coxeter. *Regular Polytopes*. Dover Publications, New York, 3rd edition, 1973.

[2] Ian Stewart. A subway named turing. *Scientific American*, 271(3):90–92, 1994. Mathematical Recreations column, the railway model of computation.

[3] Lance Pollard. Vibe: a hyperbolic tessellation, navigation, and cellular-automaton toolkit. `https://github.com/cluesurf/vibe`, 2026. Source code for the construction, the railway automaton, and the verification of this paper.

[4] Marvin L. Minsky. *Computation: Finite and Infinite Machines*. Prentice-Hall, Englewood Cliffs, NJ, 1967. Two-counter register machines are universal.

[5] Maurice Margenstern. *Cellular Automata in Hyperbolic Spaces, Volume I: Theory*. Old City Publishing, Philadelphia, 2007.

[6] Maurice Margenstern. *Cellular Automata in Hyperbolic Spaces, Volume II: Implementation and Computations*. Old City Publishing, Philadelphia, 2008.

[7] Maurice Margenstern. A new universal cellular automaton on the ternary heptagrid. *arXiv preprint*, arXiv:0903.2108, 2009.

[8] Maurice Margenstern. A weakly universal cellular automaton in the pentagrid with five states. *arXiv preprint*, arXiv:1403.2373, 2014.

[9] Maurice Margenstern. A weakly universal cellular automaton on the pentagrid with three states. *arXiv preprint*, arXiv:1510.09129, 2015.

[10] Maurice Margenstern. A weakly universal cellular automaton on the pentagrid with two states. *arXiv preprint*, arXiv:1512.07988, 2015.

[11] Maurice Margenstern. An outer totalistic weakly universal cellular automaton in the dodecagrid with four states. *arXiv preprint*, arXiv:2108.13094, 2021.

[12] Maurice Margenstern. A strongly universal cellular automaton in the dodecagrid with five states. *arXiv preprint*, arXiv:2104.01561, 2021.